



Computer Architecture
CS 325 - 001
Department of Physics and Computer Science
Medgar Evers College
Exam 3

Instructions:

- The exam requires completing a set of tasks within 120 minutes.
- For each task, write your solution in a separate file. Tasks 1 - 8 must be text files, and task 9 must be C++ file. Each file must be named 'Task0*n*' where *n* is its task number.
- The source code can only include the libraries 'iostream', 'sstream', 'fstream', 'iomanip', 'string', 'ctype', 'Object.h', 'Dev.h', 'Gate.h', 'Split.h', 'Word.h', 'Port.h', and 'Socket.h'.
- Use the IAS instruction set and Boolean algebra postulates when necessary:
- Notes are not allowed.
- Cheating of any kind is prohibited and will not be tolerated.
- Violating and/or failing to follow any of the rules will result in an automatic zero (0) for the exam.

TO ACKNOWLEDGE THAT YOU HAVE READ AND UNDERSTOOD THE INSTRUCTIONS ABOVE,
PRINT YOUR NAME AND THE DATE ON YOUR SUBMISSIONS

Grading

Section	Maximum Points	Points Earned
1	2.0	
2	2.0	
3	3.0	
4	3.0	
5	2.0	
6	2.0	
7	2.5	
8	1.5	
9	2.0	
Total	20	

1. Convert each of the following binary numbers to their 12-bit twos-complement negation. The inversion and the final outcome must be provided to receive full credit. If a binary number has fewer than 12 bits, its most significant bits are 0.

- a) 00110100
- b) 001001011000
- c) 001101
- d) 1001111001

2. Find the solution of each binary expression and state if overflow occurs. You must show work to receive full credit.

- a) $00110111 + 01011110$
- b) $11001100 - 01100011$

3. Write an IAS program, in hexadecimal, implementing the statement below, given that L, M, and N are at addresses D7, 1B, and C3, respectively.

$$L = \begin{cases} 2L + N & \text{if } N \geq M \\ 2L + M & \text{if } N < M \end{cases}$$

4. Trace your solution from Problem 1, starting from address 000, by listing all sequential changes to registers and memory, given the initial values:

Variable	Address	Initial Value
L	D7	000000001B
M	1B	000000000A
N	C3	000000000F

5. Simulate the Least Recently Used (LRU) page replacement algorithms using a cache of size 3 on the following block sequence:

(7 1 4 2 8 6 6 2 5 4 7 8 4 5 6 2 4 4 1 6)

Example (cache size 2, sequence: 2 5 2 8):

[] -> [2] -> [5,2] -> [2,5] -> [8,2]

6. Simulate the First-In-First-Out (FIFO) page replacement algorithms using a cache of size 3 on the following block sequence:

(5 8 2 8 5 1 8 2 5 3 2 3 6 3 5 4 2 7 1 7)

Example (cache size 2, sequence: 2 5 2 8):

[] -> [2] -> [2,5] -> [2,5] -> [5,8]

7. For each Boolean function (F_i) in the truth table below, write the product of sums (POS) and sum of products (SOP) representations of the function.

A	B	C	F_1	F_2
1	1	1	0	1
1	1	0	1	0
1	0	1	0	1
1	0	0	0	0
0	1	1	0	1
0	1	0	1	1
0	0	1	1	0
0	0	0	1	0

Use ' to represent NOT operation.

8. Simplify the POS circuit for Boolean function F_2 using Boolean algebra postulates below and/or a Karnaugh Map.
9. Write a cpp program that:
- Create a derived class named $F2$ that
 - Publicly inherits $Gate$.
 - Deletes its copy constructor and assignment operator.
 - Define a default constructor that makes the gate accept exactly 3 inputs.
 - Define an empty destructor.
 - Overrides `output()` to return the outcomes of F_2 from Problem 7 if valid; otherwise, it returns false.
 - Overrides `toString()` to return "T" if `output()` returns true; otherwise, it returns "F".
 - Within the main function,
 - Initialize three $Word$ objects to "11110000", "11001100", and "10101010", respectively.
 - Declare three $Port$ objects and assign them the three $Word$ objects, respectively.
 - Declare a $F2$ object, and assign the $Port$ objects as its inputs.
 - Display a truth table of the $F2$ object by displaying the $Port$ objects and the $F2$ object as you traverse through each bit of the $Word$ objects simultaneously (for each instance of the loop invoke `set()` method for each $Port$ object from 0 to 7)

IAS Instruction Set Reference

Mnemonic	Opcode	Description
LMA	0A	Transfer contents from MQ to AC
LDM	09	Transfer M(X) to MQ
STA	21	Transfer contents from AC to memory location X
LDA	01	Transfer M(X) to AC
LDN	02	Transfer -M(X) to AC
ALD	03	Transfer M(X) to AC
ALN	04	Transfer - M(X) to AC
BRL	0D	Takes next instruction from left half of M(X)
BRR	0E	Takes next instruction from right half of M(X)
BPL	0F	If AC ≥ 0 , takes next instruction from the left half of M(X)
BPR	10	If AC ≥ 0 , takes next instruction from the right half of M(X)
ADD	05	Add M(X) to AC; put result in AC
AAD	07	Add M(X) to AC; put result in AC
SUB	06	Subtract M(X) from AC; put result in AC
ASB	08	Subtract M(X) from AC; put result in AC
MUL	0B	Multiply M(X) by MQ; put most significant bits of result in AC; least significant in MQ
DIV	0C	Divide AC by M(X); put quotient in MQ and remainder in AC
LSH	14	Multiply AC by 2
RSH	15	Divide AC by 2
STL	12	Transfer AC[28:39] to M(X)[8:19]
STR	13	Transfer AC[28:39] to M(X)[28:39]
HLT	00	Halts

Boolean Algebra Postulates

Postulates		
$A \cdot B = B \cdot A$	$A + B = B + A$	Commutative Law
$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$	Associative Law
$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$	Distributive Law
$1 \cdot A = A$	$0 + A = A$	Identity Property
$A \cdot A' = 0$	$A + A' = 1$	Inverse Property
$A \cdot A = A$	$A + A = A$	Idempotent Property
$0 \cdot A = 0$	$1 + A = 1$	Annulment Property
$(A \cdot B)' = A' + B'$	$(A + B)' = A' \cdot B'$	DeMorgan's Theorem
$A \cdot (A + B) = A$	$A + (A \cdot B) = A$	Absorption Law
$A \cdot (A' + B) = A \cdot B$	$A + (A' \cdot B) = A + B$	
$A'' = A$		Double Negation Property
$A'B + AB' = A \oplus B$		Exclusive-Or Property
$AB + A'B' = (A \oplus B)'$		Exclusive-Nor Property